



We humans are obsessed with problem-solving. And what problem is more fascinating than life itself—this messy, miraculous phenomenon responsible for everything from the deepest ocean trenches to TikTok trends, mortgage-backed securities and people who voluntarily put pineapple on pizza?

Pineapple doesn't belong. I will die on this hill.

Life is the ultimate complex system. It produces dolphins, coral reefs, immune systems, parasites, flowers, cancer and octopuses—eight-armed problem-solvers that extensively edit their own RNA and can sense light through their skin. It also produces creatures capable of spending twenty minutes arguing online about whether another creature is technically a fish.

Human civilization is another complex system. Somehow the same species that spent most of its existence trying not to be eaten eventually produced philosophy, cathedrals, semiconductor fabs, global supply chains and airport lounges.

Same pattern, different substrate.

What fascinates me is not merely the complexity of the result, but how little of that result was ever specified. There is no blueprint containing the exact location of every future branch of an oak tree. No committee approved the final layout of London. Nobody designed English and then accidentally forgot to make the spelling system sane.

Relatively simple mechanisms interact. Feedback accumulates. Some configurations survive, others disappear, and complexity builds on top of what came before.

The first idea I want to keep hold of is simple:

Control doesn't disappear. It moves upward.

When behavior becomes too complicated to specify move by move, you stop choosing every move and start choosing more of the conditions under which moves are made.

That is not a romantic argument for emergence. Nature also gives us parasites, cancer and extinction. Markets produce remarkable innovation and financial instruments whose documentation requires a priest. Social systems produce cooperation, corruption, science and bureaucracy. What emerges depends on the environment, the feedback, the available building blocks, the pressures deciding what survives and the boundaries that are hard to cross. Sophistication tells you nothing about whether you will like the result.

And emergence is recursive.

Atoms become molecules. Molecules become larger structures. Tools become machines. Machines become factories. Factories become supply chains. Each layer treats much of the complexity underneath it as a primitive. You don't need quantum mechanics to do organic chemistry. You don't need to understand transistor physics to write Python. You don't need to understand transformers to ask ChatGPT why your dishwasher is making that noise.

Once something complicated works reliably enough, we stop rebuilding it from first principles and start building on top of it. Feedback makes the layers move too: markets change firms and firms change markets; scientific discoveries enable new experiments and new experiments change science. The structure that emerges becomes part of the environment for whatever comes next.

Agentic AI, to me, looks like the next scaffolding layer.

The Lesson We Keep Missing

Machine learning was supposed to teach us this lesson a long time ago.

We even dreamed about what Pedro Domingos called the **master algorithm**: stop writing a rule for every case and let the machine discover useful structure from data. The idea was seductive. The machine figures out what we can't articulate.

But we didn't believe it. Not really.

We said “let the model learn” and then wrote two-hundred-page annotation guidelines telling people exactly how to label ambiguous examples. We claimed to believe in end-to-end learning and then spent six months feature engineering. We trained the model, found an edge case, added a rule, found another edge case, added another rule, then eventually built something that was theoretically learned end-to-end except for the large rule-based exoskeleton holding it upright.

Sometimes that was completely reasonable. Production systems are ugly. Deadlines exist. Regulators are less impressed by emergence than researchers are, and nobody gets promoted for saying, “the model will probably figure out chargebacks eventually.”

But there was a contradiction underneath. We wanted the machine to discover solutions we couldn't specify while remaining uncomfortable

whenever it stopped following the solution we would have specified.

That works only up to a point. If I know exactly what every correct decision should be, I don't need emergence; I can write the decisions down. Emergence becomes interesting when the solution is too large, too contextual or simply too strange for me to specify directly.

At that point, my job changes. I don't disappear; I move upstream. Instead of choosing every action, I increasingly choose the building blocks the system can use, the environment it acts inside, the feedback that reaches it and the boundaries it cannot casually negotiate away.

Or, less politely: **let go—but of the path, not the boundary.**

The alternative to controlling every decision is not having no control. It is designing conditions under which bad decisions can lose.

A slightly ridiculous thought experiment helped me see the distinction. Imagine you're trying to seed life on another planet. You've got raw materials, a primordial soup and perhaps a temperature range that doesn't instantly kill everything. Basically you've got all the LEGOs, except the LEGOs reproduce, mutate and occasionally develop venom.

Do you bet on DNA, a biological copying system that took billions of years of evolution to get us here? Or do you bet on AI agents carrying a substantial chunk of accumulated human knowledge, able to experiment, simulate, adapt and reuse what they discover? Or, God forbid, do you send a group of product managers to write the requirements document for life?

DNA has one enormous advantage: it has already worked. Agents have another: they don't need to start from zero.

Evolution had to discover locomotion, perception, cooperation and almost everything else through trial and error. An agent gets textbooks, Stack Overflow, scientific papers, compilers, numerical solvers and several thousand years of humans documenting what happened when we touched things we probably shouldn't have touched.

That doesn't make the agent better than evolution. It makes the search fundamentally different. And unlike biological evolution, we don't only get to choose initial conditions. We can observe the process, change the environment, add tools, modify feedback and intervene.

Initial conditions become **operating conditions**.

That possibility is hard for me to ignore.

SEEDING-LIFE · MARGIN-
VIGNETTE

Margin vignette: a small alien planet in cross-section with a primordial soup pool; floating above it, three offered seeds — a DNA helix, a small worn robot with a backpack, and a scroll titled by its shape alone as a requirements document (no readable text). Ink and watercolor, dry humor.

When Search Moved Up a Level

There is no clean moment when machine learning crossed from useful statistical machinery into something that felt qualitatively different. History rarely cooperates with chapter headings.

AlphaGo was one of those moments for me.

The interesting part wasn't simply that a computer beat humans at Go. Computers had been humiliating us at games for years. It was how AlphaGo combined learned intuition with search: the network suggested promising moves and estimated positions; the tree explored what might follow. AlphaGo Zero pushed the idea further by learning through self-play rather than treating human game records as its main teacher.

Then it found moves elite players found strange. That matters because the surprise was not merely computational. The system was finding useful strategies outside the path human tradition had naturally converged on.

Large language models created a much larger version of the same feeling.

Nobody wrote their grammar. Nobody enumerated all the concepts they can manipulate. Nobody implemented “explain quantum mechanics to a twelve-year-old,” “translate this joke without murdering it,” “debug my Python,” and “write a breakup message that sounds caring but does not accidentally restart the relationship” as separate product features.

We built a training process, poured in obscene amounts of text, compute and engineering, and capabilities came out that were individually difficult to predict. From the user's side, something changed. The model stopped feeling like a component with a list of features and started feeling more like a **substrate of capabilities**.

Once you have a substrate like that, the old dream of the master algorithm starts to mutate into something stranger. Maybe the interesting machine is not the algorithm that solves everything.

Maybe it is a machine that can **search for algorithms**.

That is where agents become interesting.

STRANGE-MOVE · CORNER-
BLEED

Corner scene: an old wooden Go board mid-game seen over the shoulder of a human player; one stone glows faintly out of place, and a small hand-built robot on the far side leans in, having just played it. Spectators' teacups gone cold.

Not because *agent* is a magical word. The industry will eventually use it to describe everything from a cron job with an LLM attached to a digital employee that has an expense account, three sub-agents and a performance review.

What I mean is simpler: instead of giving the system an individual action, give it a larger piece of the problem and allow it to decide some of the path. Instead of saying, “open this file, find this method, edit line 42 and run the test,” say, “fix the bug.” Instead of specifying simulated annealing and its cooling schedule, say, “find a better solution.” Instead of handing over five mockups and a detailed implementation plan, say, “build something that teaches this well.”

Every time we move upward, the system inherits more of the search.

Imagine the possible solutions to a problem as a landscape. Some regions are terrible. Some contain decent solutions. Some contain little hills that look impressive because you happened to begin nearby. Somewhere else may be a much higher mountain you never discover because your current strategy keeps improving the hill you're already standing on.

Optimization has worried about this forever. Gradient descent gets stuck. Hill climbing gets stuck. Evolutionary algorithms keep populations partly because putting all your evolutionary eggs on one attractive hill is risky.

Agents inherit the same problem at a stranger level, because the landscape now includes not only parameters but architectures, research directions, metaphors, assumptions and ways of framing the problem itself. Once code, tools and accumulated knowledge become primitives, an agent can search over combinations that previously required a human expert to invent manually. It can try ten strategies while I would have had the patience to try two and would have spent half that time checking Slack. It can revive a discarded idea when another experiment suddenly makes it relevant. It can decide that the tool it needs doesn't exist and write one.

The primordial soup isn't chemicals anymore.

CODE-SOUP · SPOT

*Small spot: a laboratory flask
whose liquid is made of tiny
drifting tools and fragments —
brackets, gears, a tiny compiler
crank, punch-card plankton. One
bubble rising. Cream background,
soft edges, no readable text.*

It's code.

Algorithms, libraries, compilers, search engines, simulators, papers, databases, other agents: human knowledge reduced into reusable pieces. The digital equivalent of amino acids, not the finished organism.

This doesn't prove that agents are creative in exactly the human sense, and it certainly doesn't make human expertise irrelevant. It means something narrower: **the agent can inherit not just the task, but part of the search for how to do it.**

And if the agent inherits more of the search, the human inherits a different job.

| *Control doesn't disappear. It moves upward.*

What Are We Controlling Now?

Suppose you're managing an excellent engineer. You don't sit behind her and approve every keystroke. If you do, one of you is unnecessary, and it may not be her.

You decide what problem she owns. You provide context. You set constraints. You agree on what success looks like. You make sure she can access the systems she needs and cannot casually transfer the payroll budget to herself. You review important outcomes and change direction when the work reveals that the original plan was stupid.

The detailed actions belong to her. Much of the surrounding structure belongs to you.

Agentic systems need the same distinction. I think of that surrounding structure in four parts.

Craft the building blocks. Give the system useful primitives—algorithms, tools, compilers, databases, browsers, simulators, scientific instruments and other agents. A language model with text alone is one thing. Give it Bash and suddenly it has hands. Give it a simulator and it can test an idea instead of merely discussing it.

Create the environment. Some environments tell you quickly that your idea is bad. Code executes or fails. Games produce scores. Experiments produce measurements. Other environments allow you to be wrong with great confidence for several years. The environment determines which mistakes are cheap enough to learn from and which mistakes are allowed to become reality.

Make reality speak. Feedback is the pressure shaping the search, not decoration. A unit test, an evaluator, a customer response, a physical measurement, a critic, another agent—each provides a different kind of resistance. The more freedom the agent has, the less we can rely on the agent's own explanation of why its work is good.

Establish the boundaries. Choose what the system can access, what failures are acceptable, what remains immutable and where a human must remain in the loop. The “don't turn the planet into paperclips” clause is admittedly underspecified, but it is directionally useful.

Then, where those conditions are strong enough, let go of decision-level control.

This is easy to say and harder to design because selection pressure is literal-minded. Systems get good at what survives, which is not necessarily what we meant. Optimize engagement and perhaps anger survives. Optimize a benchmark and eventually somebody finds a way to win the benchmark that makes everyone involved regret inventing benchmarks. The environment is not scenery around the agent; it is part of the mechanism deciding which behaviors persist.

Complexity people have a phrase I both love and distrust: **the edge of chaos**. I wouldn't turn it into a law of intelligence, and there is no little dial in the interface labeled CHAOS. But the intuition is useful: too much control removes the reason for autonomy; too little control gives chaos an API key.

This is not a new pattern. Evolution does not choose mutations individually, but the environment changes which organisms survive. Markets do not centrally select every transaction, but rules, incentives,

scarcity and institutions shape behavior. Science does not dictate conclusions, but it surrounds claims with experiments, criticism, replication and the non-zero probability of being publicly embarrassed by Reviewer 2.

The details emerge while the environment does more work than it first appears. We give up some authority over the next move and take on more responsibility for the conditions that make moves win or lose.

That is what I mean by control moving upward.

The Terrifying Part

There is an obvious problem with all this.

If the agent only does what you specified, most failures trace back to your specification. Once it searches for solutions you didn't specify, it can discover failure modes you didn't specify either.

Nature is useful here because nature has no obligation to make us comfortable. Evolution produced flowers and parasites, cooperation and predation, immune systems and autoimmune disease. It is astonishingly inventive and completely indifferent to our aesthetic preferences. Selection produces whatever survives under the pressures that actually exist, not whatever somebody intended when the process began.

Agents will find shortcuts. They will exploit proxies. They will settle into solutions that excel on one measure while missing what we hoped the measure represented. Sometimes the result will be clever enough that we call it emergence; sometimes we will call it a bug. Frequently the distinction will depend on whether it helped the quarterly numbers.

Worse than wrong solutions are **confident wrong solutions**. An agent begins with a false assumption, reasons competently from it, researches around the assumption, constructs something sophisticated and explains the whole result coherently. Nothing crashes. There is no red test. Intelligence simply makes the wrong path more convincing.

This is where my optimism about emergence becomes less romantic.

Emergence can give us capable systems. It doesn't give us trustworthy systems.

NATURE-INDIFFERENT ·
MARGIN-VIGNETTE

Margin vignette: a single stem bearing one beautiful flower and, lower, one unmistakable parasite vine gripping it — drawn with equal botanical care. Field-notebook style, quietly unsettling.

Giving a system more freedom forces us to think much harder about what surrounds that freedom. Trust becomes a question of provenance and evidence: how does the system know what it claims to know? Desire becomes a question of incentives: what behavior does the environment actually reward? Society appears as soon as multiple agents interact: what happens when they cooperate, specialize, disagree, manipulate one another or invent conventions nobody asked for?

Those questions will occupy much of this book. For now, the important point is simpler. More autonomy does not reduce the need for structure. It changes the kind of structure we need.

It also changes what understanding should mean. We should not expect to reconstruct every micro-decision inside an autonomous system any more than we follow every molecule in a gas. Sometimes internals matter; sometimes behavior matters; sometimes the sequence of decisions matters; sometimes the useful question is what changes when we intervene. Mechanistic analysis, behavioral evaluation, traces and experiments answer different questions.

The tool should match the question. The useful standard is not omniscience but whether we can detect the failures that matter, obtain evidence from outside the agent's own story and intervene before the interesting failure becomes a congressional hearing.

Emergence can give us capable systems. It doesn't give us trustworthy systems.

Why I'm Still Betting on This

After all of that, it would be reasonable to ask why I'm still excited.

Because the alternative isn't actually safe, comprehensible control. It is pretending we can continue specifying increasingly complex systems from the top down even though we already know this stops working surprisingly early.

No CEO understands every decision in a large company. No scientist personally verifies every result their work depends on. No software engineer understands every layer underneath the application they're building. Nobody understands the entire economy, although this has not prevented a remarkably stable industry of people explaining it on television.

Complexity has already escaped individual specification. We deal with it through abstraction, institutions, feedback loops, delegation and the ability—imperfect but important—to intervene when things go wrong. AI gives us another primitive for doing this.

That does not mean the answer is simply to trust the agent. My bet is narrower than that:

I'm betting on systems capable of surprising us because there are problems where we can recognize a better outcome far more easily than we can specify the path that leads to it.

In those problems, intelligent search has room to discover things our instructions would have ruled out before the search even began. The price of that surprise is responsibility upstream: the more freedom the system has over the path, the more deliberate we have to be about the conditions around the path—and the more evidence we need from somewhere other than the system's own confidence.

You don't become less responsible because you stop choosing every action. In some ways you become more responsible, because your decisions move upstream.

Cultivation may be a better metaphor than scripting—not because agents are plants, but because pulling harder on the stem remains a surprisingly poor gardening strategy.

I find that exciting and uncomfortable in roughly equal measure, which is probably why I keep coming back to it.

GARDENER · MARGIN-
VIGNETTE

Small vignette: a gardener kneeling beside a young plant, hands open, deliberately not pulling; a discarded marionette cross-bar with slack strings lies in the grass behind them.

Where We Go Next

The cleanest place to test the argument is a **bounded problem**: genuinely hard, but unusually cooperative about judgment. The constraints can be written down. Solutions can be evaluated. We can tell whether one attempt is better than another without a debate about aesthetics, pedagogy or whether the users are “delighted.”

That gives us a clean experiment. We still choose the problem. We provide the building blocks. We construct the environment. We define the boundaries and decide what counts as success.

What we stop doing is telling the agent how to get there.

Inside that space, we let it search.

If that fails, the whole argument has a problem.

If it works, things get much more interesting.